

# Theoretical Foundations of Quantum Error-Correction

A thesis submitted to attain the degree of  
**Undergraduate in Physics**

Submitted by  
Sergio Montoya Ramirez

Supervisor  
Julian Rincon

21st May 2026

Universidad de los Andes





# Contents

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                              | <b>v</b>  |
| <b>1 Introduction</b>                                                        | <b>1</b>  |
| <b>2 Error Model</b>                                                         | <b>3</b>  |
| <b>3 Classical and Quantum Coding Theory</b>                                 | <b>9</b>  |
| Classical Codes . . . . .                                                    | 9         |
| Quantum Codes . . . . .                                                      | 9         |
| Recovery Conditions . . . . .                                                | 10        |
| Knill-Laflamme Conditions. . . . .                                           | 11        |
| Characteristics Imposed on a Code by the Knill-Laflamme Conditions . . .     | 13        |
| Degenerate Codes . . . . .                                                   | 14        |
| Quantum Hamming Bound . . . . .                                              | 14        |
| <b>4 Linear Codes</b>                                                        | <b>17</b> |
| <b>5 Symplectic (Stabilizer) Codes</b>                                       | <b>21</b> |
| Pauli operations and symplectic transformations. . . . .                     | 21        |
| Algebraic structure for $G^n$ and its relation with general operations . . . | 21        |
| Commutation. . . . .                                                         | 22        |
| Multiplication. . . . .                                                      | 23        |

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| Extending the structure . . . . .                                            | 25        |
| Unitary transformations for the extended symplectic group . . . . .          | 25        |
| Adding the symplectic structure . . . . .                                    | 25        |
| Stabilizer (Symplectic) Codes . . . . .                                      | 28        |
| <b>6 Quantum Repetition Code</b>                                             | <b>37</b> |
| Classical Repetition Code . . . . .                                          | 37        |
| Quantum Repetition Code . . . . .                                            | 39        |
| Limitations with Arbitrary Errors . . . . .                                  | 41        |
| <b>7 Shor's Code</b>                                                         | <b>43</b> |
| Code Components . . . . .                                                    | 43        |
| Standard Repetition . . . . .                                                | 43        |
| The Dual Repetition Code . . . . .                                           | 44        |
| The Complete Shor Code . . . . .                                             | 45        |
| Error Detection and Code Distance . . . . .                                  | 46        |
| $f^{(x)} \neq 0$ . . . . .                                                   | 46        |
| $f^{(x)} = 0$ (Only phase errors) . . . . .                                  | 47        |
| Concrete Example: The Original 9-Qubit Shor Code . . . . .                   | 47        |
| Shor Code as a Stabilizer Code . . . . .                                     | 48        |
| Internal Stabilizers. . . . .                                                | 49        |
| External Stabilizers . . . . .                                               | 49        |
| Dimension of the Code and Verification of the Symplectic Condition . . . . . | 50        |
| <b>8 Conclusions</b>                                                         | <b>53</b> |
| <b>Bibliography</b>                                                          | <b>55</b> |

# Abstract

Quantum computing offers an alternative with potential computational advantages for crucial algorithms; however, the inherent nature of quantum systems makes them highly sensitive to decoherence and environmental interactions. Consequently, developing robust error-correction strategies is a critical requirement for the advancement of the field. The continuous nature of quantum errors and the no-cloning theorem render classical redundancy approaches fundamentally unsuitable for quantum information. Nevertheless, the core principle of encoding information into a larger Hilbert space to protect it from decoherence remains valid and forms the theoretical foundation of quantum error-correcting codes. This thesis provides a mathematical introduction to quantum error correction, with a primary focus on the stabilizer formalism. First, a general quantum error model is formalized, followed by a detailed examination of the Knill-Laflamme conditions, which establish the necessary and sufficient criteria for a code to correct a specific set of errors. Furthermore, the theoretical bridge between classical linear codes and quantum stabilizer codes is developed utilizing the symplectic formalism. Finally, the practical application of this framework is demonstrated through a detailed analysis of the quantum repetition code and the 9-qubit Shor code, outlining their specific capabilities and limitations in fault-tolerant quantum computing.



# Introduction

Quantum computing is a promising paradigm that offers potentially exponential advantages over classical computing for problems with a specific structure, such as factorization and quantum simulation. A key property enabling these benefits is the linearity of quantum mechanics, which, in principle, prevents errors from accumulating between gates (Kitaev et al., 2002). However, the inherent quantum nature and extreme sensitivity of these systems to errors—introduced by gates or external conditions such as environmental fluctuations and electromagnetic interference—mean that linearity alone is insufficient to ensure reliable results, particularly for large-scale operations like factorization (Calderbank and Shor, 1996). Furthermore, the no-cloning theorem prohibits the copying of qubits, rendering traditional error-correction methods, such as redundancy, inapplicable in a straight forward way.

Quantum error-correction is a field developed to address this challenge. It has been demonstrated that quantum information can be encoded non-locally by distributing it across multiple qubits (Calderbank and Shor, 1996). Codes that employ this strategy are known as quantum error-correcting codes. The efficacy of this method is supported by recent experimental improvements in physical quantum computers (AbuGhanem, 2025). Nevertheless, it remains an open area of research, and advancements in this field are critical for the continued development of quantum computing (AbuGhanem, 2025).

This thesis presents the theoretical foundations of quantum error-correcting codes as a key component for fault-tolerant quantum computing. Rather than focusing on hardware implementations, this work explores the algebraic structure

and mathematical basis of the codes. It starts from the error models and fundamental limitations imposed by the underlying quantum system, culminating in the efficient formulation of the stabilizer formalism and its corresponding codes.

To this end, this thesis is structured as follows. Chapter 2 defines the error model that will permeate the subsequent chapters. Chapter 3 establishes the mathematical basis of the field, defining what a code is and the conditions it must fulfill to correct errors. Chapter 4 introduces linear codes, a classical concept that serves as inspiration for some of the most important quantum error-correcting codes. Chapter 5 defines the stabilizer formalism and stabilizer codes, a generalization of linear codes into quantum computing. Chapter 6 dives into the first code example, the repetition code, from the classical system to the quantum circuit that implements it. Chapter 7 develops the Shor code as a prime example of both a general and a stabilizer code. Lastly, Chapter 8 presents the conclusions.



# Error Model

Creating a general mathematical model for quantum error correction requires making as few assumptions as possible about the underlying error (or noise) model. By defining a framework for handling general errors, we can adapt it to any specific error type. However, for simplicity, we will still introduce a few practical assumptions. For instance, we will work within a two-dimensional Hilbert space for every qubit (while not strictly required, this simplifies the examples and conclusions; generalizations will be provided when necessary). We also broadly assume that the error is defined by an operation on the tensor product of the qubit space and an environment space. To this end, suppose we have a qubit in the state  $|\psi\rangle = \alpha_0 |0\rangle + \alpha_1 |1\rangle$  and an environment in the state  $|E\rangle$ . We define the error as the operation (Kaye et al., 2006):

$$(\alpha_0 |0\rangle + \alpha_1 |1\rangle) |E\rangle \rightarrow \alpha_0 \beta_{00} |0\rangle |E_1\rangle + \alpha_0 \beta_{01} |1\rangle |E_2\rangle + \alpha_1 \beta_{10} |0\rangle |E_3\rangle + \alpha_1 \beta_{11} |1\rangle |E_4\rangle, \quad (2.1)$$

where  $\beta_{ij}$  is the amplitude of changing the qubit state from  $|i\rangle$  to  $|j\rangle$ , as well as changing the environment in a particular way. Thus, for a general qubit, an error is an interaction that produces a new environment state (which could be different for each possible environment basis state) and changes the state of the qubit with some probability.

At first glance, modeling this seems intractable because we would be dealing with a continuous, uncountably infinite number of possible errors. However, by Theorem 2.3, we know that the set of operations  $\mathcal{B} = \{I, Z, X, XZ\}$  forms a basis for the operator space of a two-dimensional Hilbert space. This implies that every

possible error can be rewritten as a linear combination of this set. Furthermore, we can simplify our previous expression to reflect this fact (Kaye et al., 2006):

$$\begin{aligned}
& \alpha_0 \beta_1 |0\rangle |E_1\rangle + \alpha_0 \beta_2 |1\rangle |E_2\rangle + \alpha_1 \beta_3 |0\rangle |E_3\rangle + \alpha_1 \beta_4 |1\rangle |E_4\rangle \\
&= \frac{1}{2} ((\alpha_0 |0\rangle + \alpha_1 |1\rangle) (\beta_1 |E_1\rangle + \beta_3 |E_3\rangle) \\
&\quad + (\alpha_0 |0\rangle - \alpha_1 |1\rangle) (\beta_1 |E_1\rangle - \beta_3 |E_3\rangle) \\
&\quad + (\alpha_0 |1\rangle + \alpha_1 |0\rangle) (\beta_2 |E_2\rangle + \beta_4 |E_4\rangle) \\
&\quad + (\alpha_0 |1\rangle - \alpha_1 |0\rangle) (\beta_2 |E_2\rangle - \beta_4 |E_4\rangle)) \\
&= \frac{1}{2} (I |\psi\rangle (\beta_1 |E_1\rangle + \beta_3 |E_3\rangle) \\
&\quad + Z |\psi\rangle (\beta_1 |E_1\rangle - \beta_3 |E_3\rangle) \\
&\quad + X |\psi\rangle (\beta_2 |E_2\rangle + \beta_4 |E_4\rangle) \\
&\quad + ZX |\psi\rangle (\beta_2 |E_2\rangle - \beta_4 |E_4\rangle)) .
\end{aligned}$$

One important detail is that we have not expressed the new environment states in terms of a specific basis because the structure and specifications of the environment space are unknown. Moreover, while this specific expansion is only valid for a two-dimensional Hilbert space, the underlying principle—that any continuous error can be decomposed into a discrete basis of errors—holds for any finite dimension.

Generalizing this model for a state spanning multiple qubits involves representing the error as a unitary operation  $U_{\text{err}}$  acting on the joint system-environment space. This implies that:

- 1** The error can be described as a linear combination of operators acting on both the principal system and the environment.
- 2** The density matrix of the entire system after the operation is given by:

$$\rho = U_{\text{err}} |\psi\rangle |E\rangle \langle E| \langle \psi| U_{\text{err}}^\dagger. \quad (2.2)$$

- 
- 3** We can isolate the state of the principal system by calculating the partial trace over the environment:

$$\mathrm{Tr}_E(\rho) = \mathrm{Tr}_E(U_{\mathrm{err}} |\psi\rangle |E\rangle \langle E| \langle \psi| U_{\mathrm{err}}^\dagger) = \sum_{i=0}^n \varepsilon_i^Q |\psi\rangle \langle \psi| \varepsilon_i^{Q\dagger}. \quad (2.3)$$

- 4** The error channel can thus be expressed through the application of the operators  $\varepsilon_i^Q$ , which encode all the necessary information about the interaction with the environment (Kaye et al., 2006).

An important physical consideration underlying this framework is the assumption of independent errors. If the probability of an error occurring on a single qubit is  $p$ , the probability of that qubit remaining error-free is  $1 - p$ . For a state spanning multiple qubits, the probability of independent errors occurring simultaneously on two specific qubits is proportional to  $p^2$ , on three qubits it is  $p^3$ , and so on. In a realistic physical regime where the error rate is sufficiently small ( $p \ll 1$ ), these higher-order probabilities decay rapidly ( $p \gg p^2 \gg p^3$ ). Consequently, it is a highly accurate approximation to assume that the noise is heavily dominated by single-qubit errors, allowing us to focus our error correction strategies primarily on independent single-qubit faults (Nielsen and Chuang, 2010).

**Definition 2.1.** For  $n \times n$  matrices, we define the *trace inner product* (also known as the Hilbert-Schmidt inner product) as:

$$\langle A, B \rangle = \mathrm{Tr}(A^* B). \quad (2.4)$$

*Verification of properties.* We verify the properties of the Hilbert-Schmidt inner product to demonstrate its validity:

- 1** *Conjugate symmetry:*

$$\overline{\langle B, A \rangle} = \overline{\mathrm{Tr}(B^* A)} = \mathrm{Tr}((B^* A)^*) = \mathrm{Tr}(A^* B) = \langle A, B \rangle. \quad (2.5)$$

- 2** *Linearity in the second argument:* Let  $A, B, C \in \mathbb{M}(n, \mathbb{C})$  and  $\alpha, \beta \in \mathbb{C}$ :

$$\begin{aligned} \langle A, \alpha B + \beta C \rangle &= \mathrm{Tr}(A^* (\alpha B + \beta C)) = \mathrm{Tr}(\alpha A^* B + \beta A^* C) \\ &= \alpha \mathrm{Tr}(A^* B) + \beta \mathrm{Tr}(A^* C) = \alpha \langle A, B \rangle + \beta \langle A, C \rangle. \end{aligned}$$

### 3 Positive definiteness:

$$\langle A, A \rangle = \text{Tr}(A^* A) = \sum_{i=1}^n (A^* A)_{ii} = \sum_{j=1}^n \sum_{i=1}^n \bar{a}_{ji} a_{ji}. \quad (2.6)$$

For every complex number  $x$ ,  $\bar{x}x \geq 0$ , so  $\langle A, A \rangle \geq 0$ . Moreover,  $\bar{x}x = 0$  if and only if  $x = 0$ ; therefore  $\langle A, A \rangle = 0$  implies  $A = 0$ .

□

**Theorem 2.2.** *Given a set  $\mathcal{B}$  of non-zero vectors in a vector space of dimension  $n$  and an inner product  $\langle \cdot, \cdot \rangle$ ,*

$$\forall v_1, v_2 \in \mathcal{B}, v_1 \neq v_2 : \langle v_1, v_2 \rangle = 0 \implies \mathcal{B} \text{ is linearly independent.} \quad (2.7)$$

*Proof.* Let  $\sum_{i=1}^n a_i v_i = 0$ , where  $a_i$  are scalars in the underlying field, representing a linear combination. Take an arbitrary  $v_k$  and compute the inner product of both sides with  $v_k$ :

$$\left\langle v_k, \sum_{i=1}^n a_i v_i \right\rangle = \langle v_k, 0 \rangle. \quad (2.8)$$

Because  $\mathcal{B}$  is orthogonal,  $\langle v_k, v_i \rangle = 0$  for all  $i \neq k$ ; hence every term except the  $k$ -th vanishes:

$$a_k \langle v_k, v_k \rangle = 0. \quad (2.9)$$

Since  $v_k \neq 0$ , we have  $\langle v_k, v_k \rangle > 0$ , thus  $a_k = 0$ . Repeating this for each  $k$  shows that all coefficients are zero, so  $\mathcal{B}$  is linearly independent. □

**Theorem 2.3.** *The set of Pauli operations  $\mathcal{B} = \{I, Z, X, XZ\}$  forms a basis for the operators on a two-dimensional Hilbert space.*

*Proof.* Because the cardinality of  $\mathcal{B}$  equals the dimension of the operator space of the Hilbert space, it is sufficient to show that the set is linearly independent. We use the trace inner product (see Definition 2.1), which defines an inner product on  $2 \times 2$  matrices.

$$\langle I, X \rangle = \text{Tr} \left( \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}^* \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \right) = \text{Tr} \left( \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \right) = 0,$$

---


$$\begin{aligned}
\langle I, Z \rangle &= \text{Tr} \left( \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}^* \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \right) = \text{Tr} \left( \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \right) = 0, \\
\langle I, XZ \rangle &= \text{Tr} \left( \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}^* \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \right) = \text{Tr} \left( \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \right) = 0, \\
\langle X, Z \rangle &= \text{Tr} \left( \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}^* \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \right) = \text{Tr} \left( \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \right) = 0, \\
\langle X, XZ \rangle &= \text{Tr} \left( \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}^* \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \right) = \text{Tr} \left( \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \right) = 0, \\
\langle Z, XZ \rangle &= \text{Tr} \left( \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}^* \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \right) = \text{Tr} \left( \begin{pmatrix} 0 & -1 \\ -1 & 0 \end{pmatrix} \right) = 0.
\end{aligned}$$

All pairwise inner products are zero, so the set is orthogonal and therefore linearly independent. By Theorem 2.2, every operator on this Hilbert space can be represented as a linear combination of the elements of  $\mathcal{B}$ .  $\square$

This error model is crucial for understanding errors on many fronts. First, it shows that even though there are infinitely many possible errors, we only need to correct four of these for every other error to fall in line. Also, defining the errors as unitary operations makes it possible to correct them using other operations, enabling the core strategy that quantum error-correcting codes will follow.



# Classical and Quantum Coding Theory

## Classical Codes

In classical information theory, a code is defined as a subset  $M \subseteq \mathbb{B}^n$ , where  $\mathbb{B} = \mathbb{Z}/2$  is the set of classic bits, containing  $2^m$  elements (Kitaev et al., 2002). Notably,  $m$  is not restricted to integer values; for a code with six possible values,  $m = \log_2(6) \approx 2.5849$ . The difference between the physical block length  $n$  and the message length  $m$  defines the redundancy of the system. This redundancy is a fundamental requirement for verifying the integrity of a received bitstring. While these subsets typically map to external data structures, such as the ASCII standard where  $m = 7$ , they more broadly function as the essential interface between physical bit states and logical information.

In the context of error correction, codes are structured such that the underlying information remains recoverable despite noise. Classical errors are typically discrete, manifesting as “bit-flips” within a string. Consequently, techniques such as data redundancy and the cloning of information are central to classical error mitigation. It is important to note that these specific strategies are prohibited in the quantum regime due to the no-cloning theorem.

## Quantum Codes

In quantum computation, defining a code as a discrete subset of bitstrings is insufficient, as it fails to account for the continuous nature of quantum states.

Instead, a quantum code is defined as a subspace of the total Hilbert space (Kitaev et al., 2002). This formulation allows for the encoding of arbitrary superpositions of basis states. By leveraging the linearity of vector spaces, one can define an encoding transformation by mapping the original basis vectors into a linearly independent set that spans the protected code subspace.

This subspace-based approach ensures that linear transformations can be applied while the state remains within the code's boundaries. Furthermore, it provides the mathematical framework necessary to identify and rectify errors without collapsing the logical state. This is achieved by extracting an error syndrome, a diagnostic indicator of the noise process, via projective measurements onto the subspace, rather than by measuring individual physical qubits.

## Recovery Conditions

To maintain generality in the theory of quantum error-correcting codes, minimal assumptions are made regarding the noise model. We consider an error channel  $\mathcal{E}$  acting on a density matrix  $\rho$  as a trace-preserving map:

$$\mathcal{E}(\rho) = \sum_i E_i \rho E_i^\dagger. \quad (3.1)$$

where  $\{E_i\}$  are the Kraus operators representing the noise process. A code  $C$  is said to correct the channel  $\mathcal{E}$  if there exists a recovery operation  $\mathcal{R}$  such that (Nielsen and Chuang, 2010):

$$\forall \rho \in C \implies (\mathcal{R} \circ \mathcal{E})(\rho) \propto \rho. \quad (3.2)$$

The map  $\mathcal{R}$  is a physical process designed to map the corrupted state back into the code space. Critically, the recovery is considered successful if the logical information is preserved, even if the resulting physical state is not identical to the original codeword. This distinction is vital for understanding degenerate codes, where multiple distinct error processes may lead to the same corrected state within the logical subspace.



## Knill-Laflamme Conditions

The existence of a recovery operation for an error is not guaranteed for an arbitrary code. Therefore, it is crucial to have a method to determine if a code can correct an error  $\mathcal{E}$  within the subspace spanned by  $\{E_i\}$ . For a code  $\mathcal{C}$  with projector  $P$ , the necessary and sufficient condition for the code to correct the error  $\mathcal{E}$  is given by the Knill-Laflamme conditions (Nielsen and Chuang, 2010):

$$PE_i^\dagger E_j P = \alpha_{ij} P. \quad (3.3)$$

for some Hermitian matrix  $\alpha$ .

### **Proof (adapted from Nielsen and Chuang, 2010):**

To prove sufficiency, we can construct the operation  $\mathcal{R}$  assuming the Knill-Laflamme conditions are fulfilled. We assume a general method for error correction that separates syndrome measurement and error recovery, proving that both components can be generated given these conditions.

By assumption,  $\alpha$  is a Hermitian matrix and is therefore diagonalizable as  $d = u^\dagger \alpha u$ . We can define operations  $F_k := \sum_i u_{ik} E_i$ , and given Theorem 3.1,  $\{F_k\}$  is also a set of operation elements for  $\mathcal{E}$ . By substitution, we get:

$$\begin{aligned} PF_k^\dagger F_l P &= \sum_{ij} u_{ki}^\dagger u_{jl} P E_i^\dagger E_j P \\ &= \sum_{ij} u_{ki}^\dagger u_{jl} \alpha_{ij} P \\ &= \sum_{ij} u_{ki}^\dagger \alpha_{ij} u_{jl} P \\ &= d_{kl} P. \end{aligned}$$

Using the polar decomposition for  $F_k P$ , we obtain:

$$\begin{aligned} F_k P &= U_k \sqrt{(F_k P)^\dagger F_k P} \\ &= U_k \sqrt{PF_k^\dagger F_k P} \\ &= U_k \sqrt{d_{kk}} P = \sqrt{d_{kk}} U_k P. \end{aligned}$$

Therefore,  $F_k$  rotates into a subspace projected by  $P_k := U_k P U_k^\dagger = \frac{F_k P U_k^\dagger}{\sqrt{d_{kk}}}$ . Moreover, given that  $d$  is a diagonal matrix, it is evident that the spaces projected by  $P_k$  are orthogonal, allowing for a reliable syndrome measurement. This could also be augmented into a trace-preserving operation by adding the necessary subspaces (though this is not crucial in every code, such as the repetition code). The error correction step would simply involve applying  $U_k^\dagger$ .

Thus, we can construct the operation  $\mathcal{R}(\sigma) = \sum_k U_k^\dagger P_k \sigma P_k U_k$ . We can show that it satisfies the recovery condition because, without loss of generality for  $\rho \in \mathcal{C}$ :

$$\begin{aligned} \mathcal{R}(\mathcal{E}(\rho)) &= \sum_{kl} U_k^\dagger P_k F_l \rho F_l^\dagger P_k U_k \\ &= \sum_{kl} \left( U_k^\dagger P_k F_l \sqrt{\rho} \right) \left( \sqrt{\rho} F_l^\dagger P_k U_k \right) \\ &= \sum_{kl} \left( U_k^\dagger P_k F_l \sqrt{\rho} \right) \left( U_k^\dagger P_k F_l \sqrt{\rho} \right)^\dagger. \end{aligned}$$

Evaluating the inner term:

$$\begin{aligned} U_k^\dagger P_k F_l \sqrt{\rho} &= U_k^\dagger P_k F_l P \sqrt{\rho} \\ &= \frac{U_k^\dagger U_k P F_k^\dagger F_l P \sqrt{\rho}}{\sqrt{d_{kk}}} \\ &= \frac{\delta_{kl} d_{kk} P \sqrt{\rho}}{\sqrt{d_{kk}}} \\ &= \delta_{kl} \sqrt{d_{kk}} \sqrt{\rho}. \end{aligned}$$

Substituting this back yields:

$$\begin{aligned} \mathcal{R}(\mathcal{E}(\rho)) &= \sum_{kl} \left( \delta_{kl} \sqrt{d_{kk}} \sqrt{\rho} \right) \left( \delta_{kl} \sqrt{d_{kk}} \sqrt{\rho} \right)^\dagger \\ &= \sum_{kl} \delta_{kl} d_{kk} \rho \\ &\propto \rho. \end{aligned}$$

This satisfies the conditions for  $\mathcal{C}$  to correct  $\mathcal{E}$ .

To prove the necessity of the Knill-Laflamme conditions, suppose  $\mathcal{E}$  is perfectly correctable by the recovery operation  $\mathcal{R}$  with operation elements  $\{R_j\}$ . Define an operation  $\mathcal{E}_C(\rho) = \mathcal{E}(P\rho P)$  which is also recoverable by  $\mathcal{R}$ , where the proportionality factor is a constant  $c$  independent of  $\rho$ . This can be rewritten as:

$$\begin{aligned}\mathcal{R}(\mathcal{E}_C(\rho)) &= cP\rho P, \\ \sum_{ij} R_j E_i P \rho P E_i^\dagger R_j^\dagger &= cP\rho P.\end{aligned}$$

By Theorem 3.1, we know there exist complex numbers in a unitary matrix  $c$  such that:

$$\begin{aligned}R_k E_i P &= c_{ki} P \\ P E_i^\dagger R_k^\dagger &= c_{ki}^* P \\ P E_i^\dagger R_k^\dagger R_k E_j P &= c_{ki}^* c_{kj} P.\end{aligned}$$

Considering that  $\mathcal{R}$  is trace-preserving, summing over  $k$  yields:

$$P E_i^\dagger E_j P = \sum_k c_{ki}^* c_{kj} P = \alpha_{ij} P. \quad (3.4)$$

This perfectly matches the Knill-Laflamme conditions.

## Characteristics Imposed on a Code by the Knill-Laflamme Conditions

The Knill-Laflamme conditions are an elegant way to reveal that, due to the existence of  $\alpha$  as a Hermitian matrix and the conditions themselves, a quantum code that corrects an error needs to be:

- 1 Orthogonal:** The basis should be orthogonal so that the states are distinguishable, which is reflected by the syndrome measurement operation obtained in the proof of the Knill-Laflamme conditions. This is also indispensable for  $\alpha$  to be a Hermitian matrix (Kitaev et al., 2002).

- 2 Non-deformed by errors:** This corresponds, by the definition of deformation, to the Knill-Laflamme conditions. It essentially ensures that a codeword is not altered or made non-orthogonal by the application of an error. Thus, even when an error occurs, it remains possible to distinguish between states and recover them.

**Lemma 3.1** (Theorem 8.2, (Nielsen and Chuang, 2010)). *Suppose  $\{E_1, E_2, \dots, E_m\}$  and  $\{F_1, F_2, \dots, F_n\}$  are operation elements giving rise to quantum operations  $\mathcal{E}$  and  $\mathcal{F}$  respectively. By appending zero operators to the shorter list, we may ensure that  $m = n$ . Then  $\mathcal{E} = \mathcal{F}$  if and only if there exist complex numbers  $u_{ij}$  such that  $E_i = \sum_j u_{ij} F_j$ , where  $u$  is an  $m \times m$  unitary matrix.*

## Degenerate Codes

As seen in the previous sections, classical and quantum codes are considerably intertwined. However, degenerate codes are a family of quantum codes without any corresponding classical analog. In classical computing, the occurrence of errors in different bits necessarily leads to different codewords. In quantum codes, however, an error applied to different physical qubits can produce the exact same logical state (Nielsen and Chuang, 2010).

Examples of these types of codes will be introduced later. This is a crucial characteristic because, even though the exact error might not be distinguishable, its correction can still be performed using the same recovery operation. Nonetheless, it is important to consider that this distinction carries a cost: some methods used for demonstrating code properties are not applicable to this subset of codes. One relevant example is discussed in the next section regarding the quantum Hamming bound.

## Quantum Hamming Bound

Given a set of errors, there are multiple codes capable of correcting it. Therefore, for any practical application, one typically looks for a code that minimizes the size of the logical qubit. The Quantum Hamming Bound provides a way to find this lower limit for a non-degenerate quantum code.

**Definition 3.2** (Quantum Hamming Bound). Let  $C$  be a non-degenerate code encoding a  $2^k$ -dimensional logical space into a  $2^n$ -dimensional physical subspace that can correct arbitrary errors on at most  $t$  qubits. The parameters for  $C$  must satisfy the inequality:

$$\sum_{j=0}^t \binom{n}{j} 3^j 2^k \leq 2^n. \quad (3.5)$$

*Proof.* For any subset of errors occurring on  $j \leq t$  qubits, the possible combinations of error locations are given by the binomial coefficient  $\binom{n}{j}$ . For every qubit in that system, there are 3 possible non-trivial Pauli errors (since the identity operation  $I$  is discarded, as applying it does not change the state). Therefore, for every set of positions, there are  $3^j$  possible error combinations. By assumption,  $C$  is non-degenerate, meaning every combination of errors must map to an orthogonal  $2^k$ -dimensional subspace. Since  $C$  corrects every possible error configuration for  $j \leq t$ , the sum of the dimensions of all these orthogonal error subspaces cannot exceed the total dimension of the physical Hilbert space,  $2^n$  (Nielsen and Chuang, 2010).  $\square$

This bound creates a rule of thumb for minimizing the size of the logical qubit in a code. However, the lack of consideration for degenerate codes is an important caveat for the system as a whole.

**Example 3.3** (Hamming Bound for a code that corrects arbitrary errors in a single qubit). Let  $C$  be an error-correcting code capable of correcting any single-qubit error. Applying the Hamming Bound to it, we obtain:

$$\begin{aligned} \sum_{j=0}^1 \binom{n}{j} 3^j 2^k &\leq 2^n \\ \left[ \binom{n}{0} 3^0 2^1 + \binom{n}{1} 3^1 2^1 \right] &\leq 2^n \\ [2 + n \cdot 3 \cdot 2] &\leq 2^n \\ 2[1 + 3n] &\leq 2^n. \end{aligned}$$

It is apparent here that  $n$  must be equal to or greater than 5 for the inequality to hold. Therefore, the smallest non-degenerate quantum code capable of correcting arbitrary single-qubit errors requires at least 5 physical qubits. However, larger codes do exist and may offer additional benefits, such as higher fidelity when an error occurs.

# Linear Codes

Linear codes are one of the most important concepts in classical coding theory. Their efficiency in encoding information and detecting errors makes them computationally highly practical. More importantly, the general concept of linear codes is foundational to quantum error-correcting codes, specifically through their generalization into stabilizer codes. Therefore, it is important to take a brief detour from pure quantum codes to review the definition, characteristics, and advantages of linear codes, establishing the groundwork for how stabilizer codes operate.

**Definition 4.1.** A binary code  $\mathcal{C}$  that encodes  $k$  bits of information into an  $n$ -bit space is called a *linear code* if it forms a  $k$ -dimensional subspace of the vector space  $\mathbb{F}_2^n$ . Every possible  $k$ -bit string to be encoded can be represented as a column vector  $u \in \mathbb{F}_2^k$ , and its corresponding codeword  $v \in \mathcal{C}$  is generated by the operation  $v = Gu$ . Here,  $G$  is an  $n \times k$  matrix with entries in  $\mathbb{F}_2$ , called the *generator matrix*. The column vectors of  $G$  form a basis for the subspace  $\mathcal{C}$ . Thus, every operation is performed modulo two, and every valid codeword is a linear combination of these basis vectors (Gottesman, 1998).

This definition highlights how linear codes streamline the encoding process by reducing it to matrix multiplication over  $\mathbb{F}_2$ , an operation that is highly efficient to implement computationally. However, the most profound connection to stabilizer codes lies in the error detection phase, specifically through the use of a parity-check matrix.

**Lemma 4.2.** *Given a subspace  $\mathcal{C} \subseteq \mathbb{F}_2^n$  and its orthogonal complement  $\mathcal{C}^\perp$ , it holds that:*

$$(\mathcal{C}^\perp)^\perp = \mathcal{C}. \tag{4.1}$$

*Proof.* By the definition of the orthogonal complement, for any  $v \in \mathcal{C}$  and  $w \in \mathcal{C}^\perp$ , their inner product  $\langle w, v \rangle = 0$ . Therefore, every  $v \in \mathcal{C}$  is orthogonal to everything in  $\mathcal{C}^\perp$ , which implies  $\mathcal{C} \subseteq (\mathcal{C}^\perp)^\perp$ .

By calculating dimensions, we observe that:

$$\dim((\mathcal{C}^\perp)^\perp) = \dim(\mathbb{F}_2^n) - \dim(\mathcal{C}^\perp) = n - (n - \dim(\mathcal{C})) = \dim(\mathcal{C}). \quad (4.2)$$

Since  $\mathcal{C} \subseteq (\mathcal{C}^\perp)^\perp$  and both subspaces share the same dimension, they must be equal.  $\square$

**Theorem 4.3.** *Given an  $[n, k]$  linear binary code  $\mathcal{C}$  with generator matrix  $G \in \mathbb{F}_2^{n \times k}$ , there exists an  $(n - k) \times n$  matrix  $H$ , called the parity-check matrix, such that a vector  $v \in \mathbb{F}_2^n$  is a valid codeword ( $v \in \mathcal{C}$ ) if and only if  $Hv = 0$ .*

*Proof.* Since a linear code  $\mathcal{C}$  is a  $k$ -dimensional subspace of  $\mathbb{F}_2^n$ , we can consider its orthogonal complement  $\mathcal{C}^\perp \subseteq \mathbb{F}_2^n$ , defined as:

$$\mathcal{C}^\perp = \{w \in \mathbb{F}_2^n \mid \forall v \in \mathcal{C}, \langle w, v \rangle = 0\}. \quad (4.3)$$

The orthogonal complement has dimension  $\dim(\mathcal{C}^\perp) = n - k$ . Thus,  $\mathcal{C}^\perp$  has a basis formed by  $n - k$  linearly independent vectors:  $\{h_1, h_2, \dots, h_{n-k}\}$ . We construct the parity-check matrix  $H$  by setting these basis vectors as its rows:

$$H = \begin{pmatrix} - & h_1 & - \\ - & h_2 & - \\ & \vdots & \\ - & h_{n-k} & - \end{pmatrix}. \quad (4.4)$$

- (  $\implies$  ) Let  $v \in \mathcal{C}$ . By the definition of  $\mathcal{C}^\perp$ ,  $v$  is orthogonal to every basis vector  $h_i$ . Therefore, the dot product  $h_i \cdot v = 0$  for all  $i$ , which yields  $Hv = 0$ .
- (  $\impliedby$  ) Let  $v \in \mathbb{F}_2^n$  such that  $Hv = 0$ . This means that  $v$  is orthogonal to every row  $h_i$  of  $H$ . Since the rows of  $H$  span  $\mathcal{C}^\perp$ ,  $v$  is orthogonal to all of  $\mathcal{C}^\perp$ , implying that  $v \in (\mathcal{C}^\perp)^\perp$ . By Lemma 4.2, we conclude that  $v \in \mathcal{C}$ .

$\square$



---

The existence of a parity-check matrix provides a systematic framework for characterizing linear codes. During the detection phase, checking whether a received vector contains an error reduces to a simple matrix multiplication: applying the parity-check matrix to the received vector to compute the error syndrome. This process is best illustrated with a concrete example:

**Example 4.4.** The 3-bit repetition code is a simple  $[3, 1]$  linear code with generator matrix:

$$G = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}. \quad (4.5)$$

It encodes a single bit of information  $u \in \mathbb{F}_2$  into a 3-bit codeword  $v = Gu$ :

$$\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}; \quad \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}. \quad (4.6)$$

Its corresponding parity-check matrix  $H$  must satisfy  $HG = 0$ . A valid choice is:

$$H = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}. \quad (4.7)$$

When a vector  $r \in \mathbb{F}_2^3$  is received, we can check for errors by calculating the *syndrome* vector  $s = Hr$ . If  $s = 0$ , the received vector is a valid codeword. If a single-bit error occurred, the syndrome  $s$  matches one of the columns of  $H$ :

- If  $s = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ , there is no error (or an undetectable error occurred).
- If  $s = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ , error occurred in the first bit (matches the 1st column of  $H$ ).
- If  $s = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ , error occurred in the second bit (matches the 2nd column of  $H$ ).
- If  $s = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ , error occurred in the third bit (matches the 3rd column of  $H$ ).



# Symplectic (Stabilizer) Codes

## Pauli operations and symplectic transformations

In this section, we will develop the structure of a group in the Pauli operations by constructing a representation of it as a tuple and by constructing a multiplication operation between them that holds all the necessary properties of a group.

## Algebraic structure for $G^n$ and its relation with general operations

Every Pauli operation can be indexed by a tuple living in  $(\mathbb{Z}_2)^{2n}$  (Kitaev et al., 2002). Each item in the tuple represents the operation  $X$  or  $Z$ , respectively, and it is applied in case the value is 1 (Kitaev et al., 2002). However, it is necessary to add a factor of  $i$  to get the canonical Pauli operations. Therefore, we can create a homomorphism  $\sigma$  by

$$\sigma(\alpha, \beta) = i^{\alpha\beta} \sigma_{\alpha 0} \sigma_{0 \beta}. \quad (5.1)$$

with  $\sigma_{10} = X$ ,  $\sigma_{01} = Z$ , and  $\sigma_{00} = I$ . Moreover, every combination of  $n$  Pauli matrices can be represented by a vector

$$\gamma = (\alpha_1, \beta_1, \dots, \alpha_n, \beta_n) \in G^n. \quad (5.2)$$

in addition to a representation  $\sigma : G^n \rightarrow \mathcal{L}(\mathcal{B}^{\otimes n})$  that connects the  $G^n$  group to the operations over  $n$  qubits by multiplying via tensor products the result of every pair  $\sigma(\alpha_i, \beta_i)$  as defined previously (Kitaev et al., 2002). As discussed before, the operations over a 2-dimensional Hilbert space have a basis. Therefore, the space of operations  $\mathcal{L}(\mathcal{B}^{\otimes n})$  is an algebra graded by  $G^n$  (Kitaev et al., 2002); this implies that

$$\mathcal{L}(\mathcal{B}^{\otimes n}) = \bigoplus_{\gamma \in G^n} \mathbb{C}\sigma(\gamma). \quad (5.3)$$

The grading operation is preserved,  $\sigma(\gamma)^\dagger = \pm\sigma(\gamma)$ , and in particular, because we are in the Hermitian convention, it is proportional to itself. We have already defined the set corresponding to the group and its relation to the quantum gate operations. However, to correctly define a group, there is a need to implement an operation of multiplication  $G^n \times G^n \rightarrow G^n$ . Nonetheless, prior to the definition of this operation, it is crucial to define and work with a commutation operation that will be used in the multiplication operation.

## Commutation

A crucial property of Pauli operations is that they only commute or anti-commute (Kitaev et al., 2002). Therefore, the commutation of two vectors  $\gamma_1, \gamma_2 \in G^n$  corresponds to

$$\sigma(\gamma_1)\sigma(\gamma_2) = (-1)^{\omega(\gamma_1, \gamma_2)}\sigma(\gamma_2)\sigma(\gamma_1). \quad (5.4)$$

In this context,  $\omega(\gamma_1, \gamma_2)$  is equivalent to checking for every single qubit (a.k.a. for every pair  $(\alpha_i, \beta_i; \alpha'_i, \beta'_i)$ ) if they commute or anti-commute and then obtaining the parity of the summation of all operations (Kitaev et al., 2002). Moreover, every component of the summation would be equivalent to

$$\begin{aligned} \sigma(\alpha, \beta)\sigma(\alpha', \beta') &= (X^\alpha Z^\beta)(X^{\alpha'} Z^{\beta'}) \\ &= (-1)^{\beta\alpha'} (X^{\alpha+\alpha'} Z^{\beta+\beta'}) \\ \sigma(\alpha', \beta')\sigma(\alpha, \beta) &= (X^{\alpha'} Z^{\beta'})(X^\alpha Z^\beta) \\ &= (-1)^{\alpha\beta'} (X^{\alpha+\alpha'} Z^{\beta+\beta'}) \end{aligned}$$

$$\begin{aligned}\sigma(\alpha, \beta)\sigma(\alpha', \beta') &= (-1)^{\beta\alpha'}((-1)^{-\alpha\beta'}\sigma(\alpha', \beta')\sigma(\alpha, \beta)) \\ \sigma(\alpha, \beta)\sigma(\alpha', \beta') &= (-1)^{\alpha'\beta-\alpha\beta'}(\sigma(\alpha', \beta')\sigma(\alpha, \beta)).\end{aligned}$$

And adding over all qubits generalizes to

$$\omega(\gamma_1, \gamma_2) = \sum_{j=1}^n (\alpha_j \beta'_j - \alpha'_j \beta_j) \pmod{2}. \quad (5.5)$$

This  $\omega$  function now represents the parity between two Pauli operations, being 0 if they commute and 1 if they anti-commute, as these are their only options (Kitaev et al., 2002).

## Multiplication

Developing a general rule for multiplication is better done by beginning with the specific case of 1-qubit Pauli operations and generalizing from there...

$$\begin{aligned}\sigma(\alpha, \beta)\sigma(\alpha', \beta') &= (i^{\alpha\beta}\sigma(\alpha, 0)\sigma(0, \beta)) (i^{\alpha'\beta'}\sigma(\alpha', 0)\sigma(0, \beta')) \\ &= i^{\alpha\beta+\alpha'\beta'}\sigma(\alpha, 0)\sigma(0, \beta)\sigma(\alpha', 0)\sigma(0, \beta').\end{aligned}$$

With the commutation rule

$$\sigma(\gamma_1)\sigma(\gamma_2) = (-1)^{\omega(\gamma_1, \gamma_2)}\sigma(\gamma_2)\sigma(\gamma_1). \quad (5.6)$$

it is possible to commute via  $\sigma(0, \beta)\sigma(\alpha', 0) = (-1)^{\omega(0, \beta; \alpha', 0)}\sigma(\alpha', 0)\sigma(0, \beta)$  (Galindo, nd).

$$\begin{aligned}\sigma(\alpha, \beta)\sigma(\alpha', \beta') &= i^{\alpha\beta+\alpha'\beta'}\sigma(\alpha, 0) \left( (-1)^{\omega(0, \beta; \alpha', 0)}\sigma(\alpha', 0)\sigma(0, \beta) \right) \sigma(0, \beta') \\ &= i^{\alpha\beta+\alpha'\beta'}(i^2)^{\alpha'\beta}\sigma(\alpha, 0)\sigma(\alpha', 0)\sigma(0, \beta)\sigma(0, \beta') \\ &= i^{\alpha\beta+\alpha'\beta'+2\alpha'\beta}\sigma(\alpha, 0)\sigma(\alpha', 0)\sigma(0, \beta)\sigma(0, \beta').\end{aligned}$$

Remembering that  $X$  and  $Z$  are involutory operations, the result is  $\sigma(\alpha, 0)\sigma(\alpha', 0) = \sigma((\alpha \oplus \alpha'), 0)$  and that is equally valid for  $\beta$  (Galindo, nd).

$$\begin{aligned}
 \sigma(\alpha, \beta)\sigma(\alpha', \beta') &= i^{\alpha\beta+\alpha'\beta'+2\alpha'\beta}\sigma((\alpha \oplus \alpha'), 0)\sigma(0, (\beta \oplus \beta')) \\
 \sigma((\alpha \oplus \alpha'), (\beta \oplus \beta')) &= i^{(\alpha \oplus \alpha')(\beta \oplus \beta')}\sigma((\alpha \oplus \alpha'), 0)\sigma(0, (\beta \oplus \beta')) \\
 \sigma((\alpha \oplus \alpha'), 0)\sigma(0, (\beta \oplus \beta')) &= i^{-(\alpha \oplus \alpha')(\beta \oplus \beta')}\sigma((\alpha \oplus \alpha'), (\beta \oplus \beta')) \\
 \sigma(\alpha, \beta)\sigma(\alpha', \beta') &= i^{\alpha\beta+\alpha'\beta'+2\alpha'\beta}i^{-(\alpha \oplus \alpha')(\beta \oplus \beta')}\sigma((\alpha \oplus \alpha'), (\beta \oplus \beta')) \\
 \sigma(\alpha, \beta)\sigma(\alpha', \beta') &= i^{\alpha\beta+\alpha'\beta'+2\alpha'\beta-(\alpha \oplus \alpha')(\beta \oplus \beta')}\sigma((\alpha \oplus \alpha'), (\beta \oplus \beta')) \\
 \sigma(\alpha, \beta)\sigma(\alpha', \beta') &= i^{\tilde{\omega}(\alpha, \beta; \alpha', \beta')}\sigma((\alpha \oplus \alpha'), (\beta \oplus \beta')) \\
 \sigma(\alpha, \beta)\sigma(\alpha', \beta') &= i^{\tilde{\omega}(\alpha, \beta; \alpha', \beta')}\sigma(\gamma + \gamma') \\
 \tilde{\omega}(\alpha, \beta; \alpha', \beta') &= \alpha\beta + \alpha'\beta' + 2\alpha'\beta - (\alpha \oplus \alpha')(\beta \oplus \beta').
 \end{aligned}$$

One last simplification for  $\tilde{\omega}$  is performing all operations using only the fact that  $(a + b)^2 \pmod{4} = (a \oplus b)$  and also by noting that  $\forall n \in \{0, 1\}, n = n^2$  (Galindo, nd).

$$\tilde{\omega}(\alpha, \beta; \alpha', \beta') = \alpha^2\beta^2 + \alpha'^2\beta'^2 + 2\alpha'\beta - (\alpha + \alpha')^2(\beta + \beta')^2. \quad (5.7)$$

Now for  $n$  qubits, every pair of operations would be processed individually and summed after the result. Splitting by operation, we can simplify to

$$\begin{aligned}
 \sigma(\gamma_1)\sigma(\gamma_2) &= i^{\tilde{\omega}(\gamma_1; \gamma_2)}\sigma(\gamma_1 + \gamma_2) \\
 \tilde{\omega}(\gamma_1; \gamma_2) &= \tau(\gamma_1) + \tau(\gamma_2) - \tau(\gamma_1 + \gamma_2) + 2\kappa(\gamma_1, \gamma_2).
 \end{aligned}$$

This distribution takes advantage of the previous step by noting that every operation in a single  $\gamma$  (even when it is the addition of two different ones) can be separated from the only operation that involves both (Galindo, nd).

$$\begin{aligned}
 \tau(\alpha_1, \beta_1, \dots, \alpha_n, \beta_n) &= \sum_{j=1}^n \alpha_j^2 \beta_j^2, \\
 \kappa(\alpha_1, \beta_1, \dots, \alpha_n, \beta_n; \alpha'_1, \beta'_1, \dots, \alpha'_n, \beta'_n) &= \sum_{j=1}^n \alpha'_j \beta_j.
 \end{aligned}$$

## Extending the structure

In the last section, we built the necessary structure for a group in the Pauli operations by using the underlying properties of both the operations themselves and the representation of them as a tuple in  $(\mathbb{Z}_2)^{2n}$ . In this section, we will expand on the symmetries.

### Unitary transformations for the extended symplectic group

**Definition 5.1.** A unitary transformation is a superoperator of the form:

$$T(\rho) = U\rho U^\dagger. \quad (5.8)$$

In particular, for this case, we are interested in a subgroup of the unitary operations  $U$  that will preserve the graded structure of the Pauli basis. This means that we are interested in the unitary operations  $U$  that map a Pauli basis to itself, except for a phase (Kitaev et al., 2002):

$$T(\sigma(\gamma)) = U\sigma(\gamma)U^\dagger = (-1)^{v(\gamma)}\sigma(u(\gamma)). \quad (5.9)$$

Where

$$u : G^n \rightarrow G^n; \quad v : G^n \rightarrow \mathbb{Z}_2. \quad (5.10)$$

This group is called the extended symplectic group (represented by the symbol  $\text{ESp}_2(n)$ ) and it will become the basis for the stabilizers (Kitaev et al., 2002).

### Adding the symplectic structure

As you can expect from the previous definition, you can have a subset of the operations that needs to satisfy more structure. In particular, an operation  $u$  is called symplectic if the associated function  $u : G^n \rightarrow G^n$  satisfies (Kitaev et al., 2002):

- $u$  is linear over the field  $\mathbb{F}_2$  ( $u(c \cdot \gamma) = c \cdot u(\gamma) \wedge u(\gamma + \gamma') = u(\gamma) + u(\gamma')$ )

►  $u$  preserves the  $\omega$  form ( $\omega(u(\gamma); u(\gamma')) = \omega(\gamma; \gamma')$ )

The subset of symplectic transformations conforms a group called the symplectic group (represented by the symbol  $\text{Sp}_2(n)$ , as expected) (Kitaev et al., 2002). The existence of a homomorphism  $\theta : T \rightarrow u$  is immediately evident (Kitaev et al., 2002).

**Theorem 5.2.** *The homomorphism  $\theta$  has as its image the whole symplectic group ( $\text{Im } \theta = \text{Sp}_2(n)$ ) and its kernel is the Pauli operations (Kitaev et al., 2002). Therefore,*

$$E\text{Sp}_2(n)/G^n \cong \text{Sp}_2(n). \quad (5.11)$$

*Proof.* For the homomorphism  $\theta$  to be surjective, it is sufficient to show that for any  $u \in \text{Sp}_2(n)$  there exists at least one operation  $v : G^n \rightarrow \mathbb{Z}_4$  such that it absorbs the phase of the operation  $u$  in Equation 5.9 for it to preserve the multiplication rule (Galindo, nd).

$$\begin{aligned} T(\sigma(x)) &= U\sigma(x)U^\dagger = (-1)^{v(x)}\sigma(u(x)) \\ &= U(i^2)^{v(x)}\sigma(u(x)) = i^{2v(x)}\sigma(u(x)). \end{aligned}$$

Now multiplying and using the fact that  $u$  is linear (Galindo, nd):

$$\begin{aligned} T(\sigma(x))T(\sigma(y)) &= (i^{2v(x)}\sigma(u(x))) (i^{2v(y)}\sigma(u(y))) \\ &= i^{2v(x)+2v(y)}\sigma(u(x))\sigma(u(y)) \\ &= i^{2v(x)+2v(y)} (i^{\tilde{\omega}(u(x);u(y))}\sigma(u(x) + u(y))) \\ &= i^{2v(x)+2v(y)+\tilde{\omega}(u(x);u(y))}\sigma(u(x) + u(y)). \end{aligned}$$

In the other direction (Galindo, nd):

$$\begin{aligned} T(\sigma(x)\sigma(y)) &= T(i^{\tilde{\omega}(x;y)}\sigma(x+y)) \\ &= i^{\tilde{\omega}(x;y)}T(\sigma(x+y)) \\ &= i^{\tilde{\omega}(x;y)}i^{2v(x+y)}\sigma(u(x+y)) \\ &= i^{\tilde{\omega}(x;y)+2v(x+y)}\sigma(u(x+y)). \end{aligned}$$



It is clear that the only difference between these two equations is the phase. Consequently, we need to show the existence of a  $v$  that holds (Galindo, nd):

$$\begin{aligned} i^{2v(x)+2v(y)+\tilde{\omega}(u(x);u(y))} &= i^{\tilde{\omega}(x;y)+2v(x+y)} \\ 2v(x) + 2v(y) + \tilde{\omega}(u(x);u(y)) &= \tilde{\omega}(x;y) + 2v(x+y) \pmod{4} \\ 2v(x) + 2v(y) - 2v(x+y) &= \tilde{\omega}(x;y) - \tilde{\omega}(u(x);u(y)) \pmod{4} \\ 2v(x+y) - 2v(x) - 2v(y) &= \tilde{\omega}(u(x);u(y)) - \tilde{\omega}(x;y) \pmod{4}. \end{aligned}$$

Note that because  $u$  is symplectic, the commutation is preserved, and therefore the only difference is in the multiplication factor (1 or  $-1$ ), which can only be even (Galindo, nd).

$$v(x+y) - v(x) - v(y) = \frac{\tilde{\omega}(u(x);u(y)) - \tilde{\omega}(x;y)}{2} \pmod{2} = \omega(x,y). \quad (5.12)$$

It is now possible to show that this equation will always have a solution because all these equations hold as shown next (Galindo, nd):

$$\begin{aligned} \omega(y,z) - \omega(x+y,z) + \omega(x,y+z) - \omega(x,y) &= 0 \\ \frac{\tilde{\omega}(u(y);u(z)) - \tilde{\omega}(y;z)}{2} - & \\ \frac{\tilde{\omega}(u(x+y);u(z)) - \tilde{\omega}(x+y;z)}{2} + & \\ \frac{\tilde{\omega}(u(x);u(y+z)) - \tilde{\omega}(x;y+z)}{2} - & \\ \frac{\tilde{\omega}(u(x);u(y)) - \tilde{\omega}(x;y)}{2} &= 0. \end{aligned}$$

Splitting for the symplectic operation (Galindo, nd):

$$\begin{aligned} \frac{\tilde{\omega}(u(y);u(z)) - \tilde{\omega}(u(x+y);u(z)) + \tilde{\omega}(u(x);u(y+z)) - \tilde{\omega}(u(x);u(y))}{2} - & \\ \frac{\tilde{\omega}(y;z) - \tilde{\omega}(x+y;z) + \tilde{\omega}(x;y+z) - \tilde{\omega}(x;y)}{2} &= 0. \end{aligned}$$

Consider now general variables  $X, Y, Z$  that represent the vector in  $G^n$  or the vector after applying  $u$  (Galindo, nd).

$$\begin{aligned}
 (\sigma(X)\sigma(Y))\sigma(Z) &= (i^{\tilde{\omega}(X,Y)}\sigma(X+Y))\sigma(Z) \\
 &= i^{\tilde{\omega}(X,Y)+\tilde{\omega}(X+Y,Z)}\sigma(X+Y+Z) \\
 \sigma(X)(\sigma(Y)\sigma(Z)) &= \sigma(X)(i^{\tilde{\omega}(Y,Z)}\sigma(Y+Z)) \\
 &= i^{\tilde{\omega}(Y,Z)+\tilde{\omega}(X,Y+Z)}\sigma(X+Y+Z) \\
 (\sigma(X)\sigma(Y))\sigma(Z) &= \sigma(X)(\sigma(Y)\sigma(Z)) \\
 i^{\tilde{\omega}(X,Y)+\tilde{\omega}(X+Y,Z)}\sigma(X+Y+Z) &= i^{\tilde{\omega}(Y,Z)+\tilde{\omega}(X,Y+Z)}\sigma(X+Y+Z) \\
 \tilde{\omega}(X, Y) + \tilde{\omega}(X+Y, Z) &= \tilde{\omega}(Y, Z) + \tilde{\omega}(X, Y+Z) \\
 \tilde{\omega}(X, Y) - \tilde{\omega}(Y, Z) + \tilde{\omega}(X+Y, Z) - \tilde{\omega}(X, Y+Z) &= 0.
 \end{aligned}$$

Therefore every section is equal to 0 (Galindo, nd).

$$\begin{aligned}
 \omega(x, y) &= \omega(y, x) \\
 \omega(x, x) &= 0.
 \end{aligned}$$

Both of these come from the symplectic origin of  $u$  conserving the commutation operation and therefore fulfilling both needs (Kitaev et al., 2002). These equations are the necessary conditions for the equation to be cocyclic and therefore a solution exists for all  $u$  (Galindo, nd). This means that no matter what symplectic operation is chosen, there is a unitary operation in the extended symplectic group that implements it (Kitaev et al., 2002). This has as an immediate consequence that every symplectic operation can be constructed in a quantum circuit (Kitaev et al., 2002).  $\square$

## Stabilizer (Symplectic) Codes

Stabilizer codes (or symplectic codes) are the analogues of linear codes for quantum computing (Kitaev et al., 2002). In particular, they develop via the symplectic group (and its corresponding extended symplectic operation) similar properties to the classical parity-check matrix for linear codes (Kitaev et al., 2002).

**Definition 5.3** (Stabilizer Code). A symplectic quantum code is a subspace of the form (Kitaev et al., 2002):

$$\mathcal{M} = \{|\varepsilon\rangle \in \mathcal{B}^{\otimes n} : \forall j, X_j |\varepsilon\rangle = |\varepsilon\rangle\}, \quad (5.13)$$

with  $X_j = (-1)^{\mu_j} \sigma(f_j)$ ,  $f_j \in G^n$ ,  $\mu_j \in \mathbb{Z}_2$  (Kitaev et al., 2002).

Note that it is necessary that all the operations  $X_j$  commute; if this weren't the case, a set of mutual eigenvectors could not exist (Kitaev et al., 2002). Nonetheless, not every symplectic operation that commutes with those defined by a code must be considered for the existence and characteristics of a stabilizer code. From there, the existence of an isotropic space must follow.

**Definition 5.4** (Isotropic Space). A subspace  $F_+ \subseteq G^n$  is called Isotropic if (Kitaev et al., 2002):

$$\forall f, g \in F_+, \omega(f, g) = 0. \quad (5.14)$$

Therefore, every stabilizer code is defined by a subset  $F \subseteq F_+$  of an isotropic space (Kitaev et al., 2002). Moreover, the code depends exclusively on the subspace  $F \subseteq F_+$  spanned by the vectors  $f_j$  and the function  $\mu : F \rightarrow \mathbb{Z}_2$  such that the following equation holds (Galindo, nd):

$$\begin{aligned} \forall f, g \in F, X_\mu(f+g) &= X_\mu(f)X_\mu(g) \\ &= ((-1)^{\mu(f)}\sigma(f))((-1)^{\mu(g)}\sigma(g)) \\ &= (-1)^{\mu(f)+\mu(g)}i^{\tilde{\omega}(f,g)}\sigma(f+g) \\ X_\mu(f+g) &= (-1)^{\mu(f)+\mu(g)}(-1)^{\frac{\tilde{\omega}(f,g)}{2}}(-1)^{-\mu(f+g)}X_\mu(f+g) \\ X_\mu(f+g) &= (-1)^{\mu(f)+\mu(g)+\frac{\tilde{\omega}(f,g)}{2}-\mu(f+g)}X_\mu(f+g). \end{aligned}$$

Because they need to commute, it must follow that (Galindo, nd):

$$\begin{aligned} \mu(f) + \mu(g) + \frac{\tilde{\omega}(f, g)}{2} - \mu(f + g) &= 0 \\ \mu(f + g) - \mu(f) - \mu(g) &= \frac{\tilde{\omega}(f, g)}{2}. \end{aligned}$$

From these equations, we can deduce that  $v(f, g) = \frac{\tilde{\omega}(f, g)}{2}$  also holds the conditions for cocyclic solutions, and therefore a solution to that function always exists (Galindo, nd).

- The function is simmetric

$$\begin{aligned} v(f, g) &= \mu(f + g) - \mu(f) - \mu(g) \\ v(f, g) &= \mu(g + f) - \mu(g) - \mu(f) \\ v(f, g) &= v(g, f). \end{aligned}$$

- Before we present the cocyclic expression as a lemma, we must prove that (Galindo, nd):

$$\tilde{\omega}(f + g, h) = \tilde{\omega}(f, h) + \tilde{\omega}(g, h) + 2\chi(f, g, h). \quad (5.15)$$

For this purpose we can develop as

$$\begin{aligned} \tilde{\omega}(f + g, h) &= \tau(f + g) + \tau(h) - \tau(f + g + h) + 2\chi(f + g, h) \\ \tilde{\omega}(f + g, h) &= \tau(f + g) + \tau(h) - \tau(f + g + h) + 2\chi(f, h) + 2\chi(g, h) \\ \tilde{\omega}(f, h) + \tilde{\omega}(g, h) &= \tau(f) + \tau(g) + 2\tau(h) - \tau(f + h) - \tau(g + h) + 2\chi(f, h) + 2\chi(g, h). \end{aligned}$$

With that, we can now calculate the difference (Galindo, nd):

$$\begin{aligned} \tilde{\omega}(f + g, h) - [\tilde{\omega}(f, h) + \tilde{\omega}(g, h)] &= \tau(f + g) + \tau(h) - \tau(f + g + h) + 2\chi(f, h) + 2\chi(g, h) \\ &\quad - [\tau(f) + \tau(g) + 2\tau(h) - \tau(f + h) - \tau(g + h) \\ &\quad + 2\chi(f, h) + 2\chi(g, h)] \\ \tilde{\omega}(f + g, h) - [\tilde{\omega}(f, h) + \tilde{\omega}(g, h)] &= \tau(f + g) + \tau(h) - \tau(f + g + h) \\ &\quad - [\tau(f) + \tau(g) + 2\tau(h) - \tau(f + h) - \tau(g + h)]. \end{aligned}$$

Because  $\omega(f, g) = 0$  and  $\tilde{\omega} \equiv \omega \pmod{2}$ , therefore  $\tilde{\omega} \in \{0, 2\}$ , and thus we can define the function (Galindo, nd):

$$2\chi(f, g, h) = \tau(f + g) + \tau(h) - \tau(f + g + h) - [\tau(f) + \tau(g) + 2\tau(h) - \tau(f + h) - \tau(g + h)]. \quad (5.16)$$

And with this, we prove the lemma needed (Galindo, nd) and can continue with the conditions:

$$\begin{aligned}
 v(f + g, h) - v(f, h) - v(g, h) &= \frac{1}{2} [\tilde{\omega}(f + g, h) - \tilde{\omega}(f, h) - \tilde{\omega}(g, h)] \\
 \tilde{\omega}(f + g, h) &= \tilde{\omega}(f, h) + \tilde{\omega}(g, h) + 2\kappa(f, g, h) \\
 v(f + g, h) - v(f, h) - v(g, h) &= \frac{1}{2} [\tilde{\omega}(f, h) + \tilde{\omega}(g, h) + 2\kappa(f, g, h) - \tilde{\omega}(f, h) - \tilde{\omega}(g, h)] \\
 v(f + g, h) - v(f, h) - v(g, h) &= \kappa(f, g, h).
 \end{aligned}$$

And because every  $f, g, h \in F$  commutes (Kitaev et al., 2002):

$$v(f + g, h) - v(f, h) - v(g, h) = 0. \quad (5.17)$$

► finally  $v$  is 0 when applied to the same operator because

$$\begin{aligned}
 \tilde{\omega}(f, f) &= \tau(f) + \tau(f) - \tau(f + f) + 2\kappa(f, f) \\
 &= 0.
 \end{aligned}$$

And therefore (Galindo, nd):

$$v(f, f) = 0. \quad (5.18)$$

This way we demonstrate that there is always a solution for  $v(f, g)$ , but there are still two important conclusions:

- 1** For two solutions  $\mu_1$  and  $\mu_2$ , the function  $\lambda(f) = \mu_1(f) - \mu_2(f)$  is linear (Galindo, nd):

$$\begin{aligned}
 \lambda(f + g) &= \mu_1(f + g) - \mu_2(f + g) \\
 \lambda(f + g) &= [\mu_1(f) + \mu_1(g)] - [\mu_2(f) + \mu_2(g) + v(f, g)] \\
 \lambda(f + g) &= [\mu_1(f) - \mu_2(f)] + [\mu_1(g) - \mu_2(g)] \\
 \lambda(f + g) &= \lambda(f) + \lambda(g).
 \end{aligned}$$

- 2** We can define an affine space with the operation  $\lambda$ , and the dimension is the same as in the isotropic subspace  $F$  (Galindo, nd). Therefore, given that we

are working with two possible values, the complete space has (Kitaev et al., 2002):

$$|F^*| = 2^{\dim F} = 2^s. \quad (5.19)$$

It is now possible to define an abstract stabilizer code for every isotropic subspace  $F$  and function  $\mu$  (Kitaev et al., 2002):

$$\text{SympCode}(F, \mu) := \{|\psi\rangle \in \mathcal{B}^{\otimes n} : \forall f \in F, \sigma(f)|\psi\rangle = (-1)^{\mu(f)}|\psi\rangle\}. \quad (5.20)$$

**Theorem 5.5** (Orthogonal Decomposition). *Let  $F \subseteq G^n$  be an isotropic subspace of dimension  $s$  (Kitaev et al., 2002). Then it holds that:*

- 1** Every code has a dimension of (Kitaev et al., 2002):

$$\dim(\text{SympCode}(F, \mu)) = 2^{n-s}. \quad (5.21)$$

- 2** The congruent codes of  $F$  (meaning varying  $\mu$  over every solution of the  $2^s$ ) form an orthogonal decomposition of the total space of qubits (Kitaev et al., 2002):

$$\mathcal{B}^{\otimes n} = \bigoplus_{\mu} \text{SympCode}(F, \mu). \quad (5.22)$$

*Proof.* **1** The exact dimension can be calculated using the trace of the projector onto the symplectic code (Kitaev et al., 2002). For a code  $\text{SympCode}(F, \mu)$  with operators  $\{M_1, M_2, \dots, M_s\}$ , we can construct the projector (Kitaev et al., 2002):

$$P_{\mu} = \prod_{i=1}^s \frac{I + \mu_i M_i}{2}. \quad (5.23)$$

To show that it is a projector (Galindo, nd):

$$\begin{aligned} P_{i, \mu_i}^2 &= \left( \frac{I + \mu_i M_i}{2} \right) \left( \frac{I + \mu_i M_i}{2} \right) \\ &= \frac{1}{4} (I^2 + 2\mu_i M_i + \mu_i^2 M_i^2) \\ &= \frac{1}{4} (I + 2\mu_i M_i + I) \\ &= \frac{1}{4} (2I + 2\mu_i M_i) \end{aligned}$$

$$P_{i,\mu_i}^2 = \frac{I + \mu_i M_i}{2}$$

$$P_{i,\mu_i}^2 = P_{i,\mu_i}.$$

Because the square of every term corresponds to itself, the square of the product is the product itself (Galindo, nd). And to show that it projects to  $\text{SympCode}(F, \mu)$ , it is necessary to show that every vector projected by  $P_\mu$  is in the code (Galindo, nd). Without loss of generality, for a concrete stabilizer  $M_k$  and the general projector, we get (Galindo, nd):

$$M_k P_\mu |\psi\rangle = M_k \left( \prod_{i=1}^s \frac{I + \mu_i M_i}{2} \right) |\psi\rangle. \quad (5.24)$$

Because every stabilizer commutes with every other, the operator  $M_k$  can be factored into the term  $P_{k,\mu_k}$  (Kitaev et al., 2002).

$$\begin{aligned} M_k P_\mu |\psi\rangle &= \left( \prod_{i \neq k}^s \frac{I + \mu_i M_i}{2} \right) \left( M_k \left( \frac{I + \mu_k M_k}{2} \right) \right) |\psi\rangle \\ &= \left( \prod_{i \neq k}^s \frac{I + \mu_i M_i}{2} \right) \left( \frac{M_k + \mu_k M_k^2}{2} \right) |\psi\rangle \\ &= \left( \prod_{i \neq k}^s \frac{I + \mu_i M_i}{2} \right) \left( \frac{M_k + \mu_k I}{2} \right) |\psi\rangle \\ &= \left( \prod_{i \neq k}^s \frac{I + \mu_i M_i}{2} \right) \mu_k \left( \frac{\mu_k M_k + I}{2} \right) |\psi\rangle. \end{aligned}$$

Concluding therefore

$$M_k P_\mu |\psi\rangle = \mu_k \left( \prod_{i=1}^s \frac{I + \mu_i M_i}{2} \right) |\psi\rangle. \quad (5.25)$$

Given that we have already shown that  $P_\mu$  is a projector to  $\text{SympCode}(F, \mu)$ , it is therefore sufficient to get the trace as (Galindo, nd):

$$P_\mu = \frac{1}{2^s} (I + \mu_1 M_1)(I + \mu_2 M_2) \dots (I + \mu_s M_s). \quad (5.26)$$

Because we are in a closed group, the multiplication of these  $s$  elements is equal to (Galindo, nd):

$$P_\mu = \frac{1}{2^s} \left( I + \sum_{g \in F \setminus \{I\}} c_g g \right). \quad (5.27)$$

And because for every non-identity Pauli operator  $\text{Tr}(g) = 0$ , the only important trace is  $\text{Tr}(I) = 2^n$ , and therefore (Kitaev et al., 2002):

$$\text{Tr}(P_\mu) = 2^{n-s}. \quad (5.28)$$

- 2** Let  $|\psi\rangle \in \text{SympCode}(F, \mu)$  and  $|\phi\rangle \in \text{SympCode}(F, \mu')$  with  $\mu \neq \mu'$  (Galindo, nd). Therefore, for at least one  $k$ , without loss of generality,  $\mu_k = 1 \wedge \mu'_k = -1$ , and therefore (Galindo, nd):

$$\begin{aligned} \langle \phi | M_k | \psi \rangle &= \langle \phi | 1 | \psi \rangle \\ &= \langle \phi | \psi | \phi | \psi \rangle \\ \langle \phi | M_k | \psi \rangle &= (\langle \phi | M_k^\dagger) | \psi \rangle \\ &= (M_k | \phi \rangle)^\dagger | \psi \rangle \\ &= (-1 | \phi \rangle)^\dagger | \psi \rangle \\ &= - \langle \phi | \psi | \phi | \psi \rangle. \end{aligned}$$

Equating both results (Galindo, nd):

$$\begin{aligned} \langle \phi | \psi | \phi | \psi \rangle &= - \langle \phi | \psi | \phi | \psi \rangle \\ \langle \phi | \psi | \phi | \psi \rangle &= 0. \end{aligned}$$

And therefore, every code  $\text{SympCode}(F, \mu)$  is orthogonal to every other code in the same isotropic subspace (Galindo, nd). Given that we already know that it has a dimension of  $2^{n-s}$  and that we have  $2^s$  orthogonal solutions to  $\mu$ , then the direct sum of every symplectic code in  $F$  would have dimension  $2^n$  and correspond to the whole qubit space (Kitaev et al., 2002).

□



This last theorem and the general definition of the abstract stabilizer code are the only things needed to show that it corrects some errors (Kitaev et al., 2002). Let  $|\psi\rangle = \sigma(g) |\varepsilon\rangle$  be a state where an error  $\sigma(g)$  is applied to the state  $|\varepsilon\rangle$ , which is an element of a code  $\text{SympCode}(F, \mu)$  (Kitaev et al., 2002). It is true that this error must be a Pauli operation, but as said before, the space of operations over  $n$  qubits is a graded algebra of the Pauli operations. Therefore, the result of this new state with an error when a stabilizer is applied would be (Kitaev et al., 2002):

$$\begin{aligned}
 \sigma(f) |\psi\rangle &= \sigma(f) \sigma(g) |\varepsilon\rangle \\
 &= (-1)^{\omega(f,g)} \sigma(g) \sigma(f) |\varepsilon\rangle \\
 &= (-1)^{\omega(f,g) + \mu(f)} \sigma(g) |\varepsilon\rangle \\
 &= (-1)^{\omega(f,g) + \mu(f)} |\psi\rangle \\
 &= (-1)^{\mu'(f)} |\psi\rangle.
 \end{aligned}$$

There are 3 possibilities in this section:

- 1**  $\omega(f, g) \neq 0$ : This implies that  $f$  and  $g$  do not commute, meaning  $g$  is not in the centralizer of  $F$ , and therefore the error is detected by having an opposite phase (Kitaev et al., 2002). Because the space of  $n$  qubits is orthogonally decomposed by the symplectic codes of  $F$ , it is possible to project this state (because we know the new  $\mu'$  value now) to the code used and therefore correct the error (Kitaev et al., 2002).
- 2**  $\omega(f, g) = 0 \wedge g \in F$ : This is indistinguishable from applying an identity operation (varying perhaps by a phase that is expected from applying the operation but conserving the result as expected) (Kitaev et al., 2002). Therefore, it is considered that this state has not suffered an error (Kitaev et al., 2002).
- 3**  $\omega(f, g) = 0 \wedge g \notin F$ : This means that  $f$  and  $g$  commute, but the subspace that defines the code does not consider this particular error (Kitaev et al., 2002). The error maps the state to another congruent code to the original code (Kitaev et al., 2002).

**Definition 5.6** (Distance in Symplectic Codes). As stated before, the undetectable/uncorrectable errors in a symplectic code are well defined via the complement subspace of the isotropic subspace  $F$  that defines the abstract code (Kitaev et al., 2002). In simpler terms:

For an isotropic space  $F_+$  and  $F \subseteq F_+$ , a code  $\mathcal{M} = \text{SympCode}(F, \mu)$  has a distance of (Kitaev et al., 2002):

$$d(\mathcal{M}) = \min \{|f| : f \in F_+ \setminus F\}. \quad (5.29)$$

# Quantum Repetition Code

## Classical Repetition Code

Suppose a noisy channel exists in which a bit flips with a probability  $p$  (Nielsen and Chuang, 2010). This implies that we will obtain the correct information with a probability of only  $(1 - p)$ . We can achieve better results by encoding the information. In this case, let us consider taking a single bit of information and encoding it into three bits. A particular aspect of this model is that every bit in the resulting sequence holds the same amount of information. This is a strictly classical concept known as *redundancy* (Nielsen and Chuang, 2010).

$$\begin{aligned} 1 &\rightarrow 111 \\ 0 &\rightarrow 000 \end{aligned} \tag{6.1}$$

After passing through the channel, we obtain a new bit string, which for the sake of this example we will call  $S_n$ . This string could match the original or result in any other combination of the three bits, depending on the number of errors that occurred. Decoding is where the primary advantages of using a code become apparent. In this example, there are two main decoding methods to improve fidelity (Nielsen and Chuang, 2010):

- 1** We could implement a majority-vote system, meaning we interpret the bit string as representing bit 0 if the majority of the bits are 0, and similarly for 1 (Nielsen and Chuang, 2010).

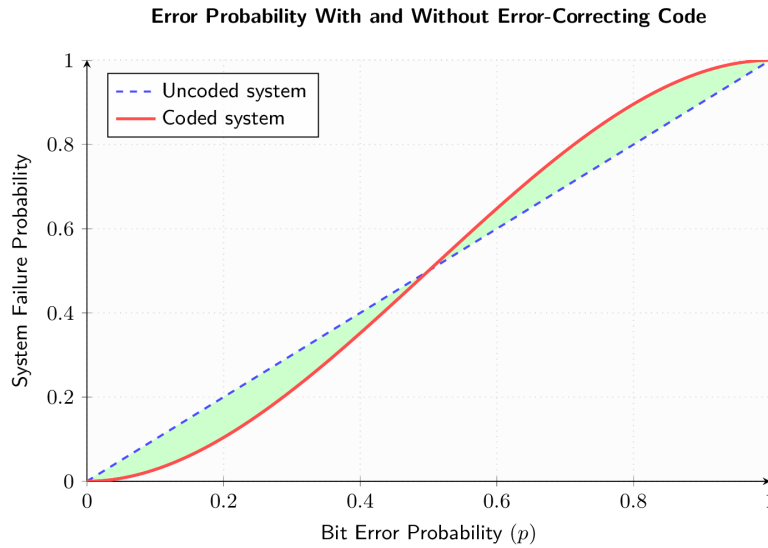
- 2** Alternatively, we can check if pairs of bits are equal. This detects whether an error occurred and indicates how to correct it. In this specific scenario, one can compare the first bit with the second, and the first bit with the third (Nielsen and Chuang, 2010).

These two systems function essentially the same way within classical computing theory. However, the first method cannot be directly translated into quantum computing because it requires measuring each qubit independently. Therefore, we will focus on the second option for the decoding process to draw a direct parallel with the quantum equivalent of this code (Nielsen and Chuang, 2010).

For ease of understanding, consider the case where the decoding process receives the bit string 010. In this scenario, the decoding procedure indicates that the first and second bits are different, while the first and third bits are equal, implying that the second bit was the source of the error (Nielsen and Chuang, 2010).

However, that assumption could be incorrect; it is entirely possible that the first and third bits were the ones affected by the error. This indistinguishability between the two possibilities demonstrates that this code only works effectively if an error occurs in at most one bit. Consequently, the probability of an error remaining uncorrected with this code is (Nielsen and Chuang, 2010):

$$p^3 + 3p^2(1 - p) = 3p^2 - 2p^3. \tag{6.2}$$



**Figure 6.1:** Comparison of the error probability for a bit transmitted through the channel with and without the code (Nielsen and Chuang, 2010). The dashed blue line represents the unprotected bit, whose error probability is linear. The red line represents the encoded system, which performs better when the physical error probability is less than 50%.

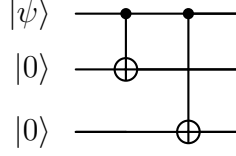
Note that without encoding, the probability of failure is simply  $p$ . Therefore, this code is only helpful if  $3p^2 - 2p^3 \leq p$ , which holds exclusively when  $p \leq \frac{1}{2}$  (Nielsen and Chuang, 2010). One can increase the number of bits used to encode the same information to obtain different thresholds of utility—a topic extensively studied in classical coding theory. Nevertheless, the core idea of encoding information is already captured by this three-bit example (Nielsen and Chuang, 2010).

## Quantum Repetition Code

Having reviewed the classical repetition code in Section 6, we can now transition to its quantum counterpart. This code relies on essentially the same theoretical principle. However, the decoding process must be interpreted through a different lens: we cannot duplicate identical information across multiple qubits due to quantum constraints. Instead, the information is distributed across all three qubits (Nielsen and Chuang, 2010).

The first defining characteristic of this code is its encoding process. In a classical code, a bit can simply be copied; however, this is impossible in quantum

computing due to the no-cloning theorem. Nonetheless, we can construct a unitary operator that encodes a single logical qubit into three physical qubits, as illustrated by the circuit in Figure 6.2 (Nielsen and Chuang, 2010).



**Figure 6.2:** Circuit that encodes a single qubit in a manner equivalent to the classical repetition code. This demonstrates that encoding is possible due to the existence of a linear, unitary process (Nielsen and Chuang, 2010).

Once encoded, the logical codespace is defined by the following projector (Nielsen and Chuang, 2010):

$$P = |000\rangle \langle 000| + |111\rangle \langle 111|. \quad (6.3)$$

The decoding process, as demonstrated previously, is divided into two stages. First, we extract the error syndrome, which is achieved by applying the following projectors (Nielsen and Chuang, 2010):

- $P_0 = |000\rangle \langle 000| + |111\rangle \langle 111|$  (No error occurred)
- $P_1 = |100\rangle \langle 100| + \dots + |011\rangle \langle 011|$  (Error in the first qubit)
- $P_2 = |010\rangle \langle 010| + \dots + |101\rangle \langle 101|$  (Error in the second qubit)
- $P_3 = |001\rangle \langle 001| + \dots + |110\rangle \langle 110|$  (Error in the third qubit)

Second, we correct the error by applying the appropriate recovery operation based on the syndrome obtained. This approach is analogous to the classical problem; therefore, the fidelity analysis is identical and yields equivalent results (Nielsen and Chuang, 2010).

Furthermore, because this is a quantum code, we can explicitly demonstrate its ability to correct bit-flip errors using the Knill-Laflamme conditions (Nielsen and Chuang, 2010). Given our previously defined projector  $P$  and the set of single-qubit bit-flip error operators:

$$\{E_i\} = \{I \otimes I \otimes I, X \otimes I \otimes I, I \otimes X \otimes I, I \otimes I \otimes X\}. \quad (6.4)$$

We can now verify that the Knill-Laflamme conditions hold for this code. First, considering the cases where  $i = j$  (since  $E_i^\dagger E_i = I$  for Pauli matrices), the condition reduces to:

$$\begin{aligned} P E_i^\dagger E_i P &= P I P \\ &= P^2 \\ &= P. \end{aligned}$$

Therefore,  $\alpha_{ii} = 1$ . Conversely, for  $i \neq j$ , applying the error operators to the projector  $P$  yields  $P_{ij}$ , which is the original operator modified by the application of  $E_i$  and  $E_j^\dagger$ . This resulting operator is orthogonal to the original projector, meaning  $\alpha_{ij} = 0$ . In summary, the matrix  $\alpha$  is the identity matrix, which is Hermitian, as required (Nielsen and Chuang, 2010).

## Limitations with Arbitrary Errors

Let us consider the standard phase-basis states (Nielsen and Chuang, 2010):

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}; \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}. \quad (6.5)$$

Now, suppose the operations  $I$  and  $Z$  are applied exclusively to the first qubit as the errors for the Knill-Laflamme conditions. Focusing solely on the first qubit, we observe that (Nielsen and Chuang, 2010):

$$\begin{aligned} \langle - | + | - | + \rangle &= 0 \\ \langle - | I^\dagger Z | + \rangle &= \langle - | - | - | - \rangle \neq 0. \end{aligned}$$

Consequently, it is impossible to correct phase-flip errors using this specific repetition code (Nielsen and Chuang, 2010).





# Shor's Code

The Shor code was the first quantum error-correcting code, described by Peter Shor and A. R. Calderbank in 1996 in the paper *Good Quantum Error-Correcting Codes Exist*. However, the original definition was later generalized into an  $n^2$ -qubit subspace that corrects general errors with a distance of  $n$  (Galindo, nd). The general idea behind such a code is to encode the information using a repetition code across two orthogonal bases. This way, any error applied to the code can be detected and corrected through projection. Furthermore, the Shor code is the first example of a symplectic code, offering all the associated advantages (Kitaev et al., 2002).

## Code Components

### Standard Repetition

The first code that will be used is the standard repetition code, as seen in Chapter 6 (Kitaev et al., 2002). This code immediately protects against bit-flip errors. However, we have shown that the 3-qubit repetition code is insufficient for the completely general case. Therefore, we generalize the repetition code to  $n$  qubits using  $V_n^z$ :

$$V_n^z : |a\rangle \rightarrow |a\rangle \otimes |a\rangle \otimes \cdots \otimes |a\rangle = |a\rangle^{\otimes n}. \quad (7.1)$$

(Galindo, nd) However, the change in code size does not affect the insufficiency to correct phase-flip errors, as discussed in the section on errors for the repetition code.

## The Dual Repetition Code

We know that phase errors and bit-flip errors are mutually conjugate (Kitaev et al., 2002). Therefore, we can expand the repetition code into a dual repetition code by encoding the new qubits into the conjugate basis using:

$$|\eta_b\rangle = H |b\rangle. \quad (7.2)$$

Encoding a qubit into  $n$  qubits with the operation  $V_n^x$  yields:

$$V_n^x : |\eta_a\rangle \rightarrow |\eta_a\rangle \otimes |\eta_a\rangle \otimes \cdots \otimes |\eta_a\rangle = |\eta_a\rangle^{\otimes n}. \quad (7.3)$$

Borrowing the proof for the correction of bit-flip errors in the repetition code, we can apply the same logic here, considering that the errors are conjugate in the Hadamard basis.

Finally, to see the complete encoding for this code, we can decompose it as:

$$\begin{aligned} |a\rangle &= \frac{1}{\sqrt{2}} \sum_{b \in \{0,1\}} (-1)^{ab} |\eta_b\rangle \\ V_n^x |a\rangle &= \frac{1}{\sqrt{2}} \sum_{b \in \{0,1\}} (-1)^{ab} |\eta_b\rangle^{\otimes n} \\ V_n^x |a\rangle &= \frac{1}{\sqrt{2}} \sum_{b \in \{0,1\}} (-1)^{ab} \left( \frac{1}{2^{n/2}} \sum_{y \in \mathcal{B}^n} (-1)^{b(\sum_{i=1}^n y_i)} |y_1, \dots, y_n\rangle \right) \\ V_n^x |a\rangle &= \frac{1}{2^{(n+1)/2}} \sum_{y \in \mathcal{B}^n} |y\rangle \left( \sum_{b \in \{0,1\}} (-1)^{b(a + \sum y_i)} \right). \end{aligned}$$

Considering exclusively the term in parentheses:

- If  $a + \sum y_i$  is even, then  $(-1)^0 + (-1)^0 = 2$ .

► If  $a + \sum y_i$  is odd, then  $(-1)^0 + (-1)^1 = 0$ .

This factor of 2 cancels part of the denominator:

$$\frac{2}{2^{(n+1)/2}} = 2^{1-(n+1)/2} = 2^{-(n-1)/2}.$$

Thus, the complete equation becomes:

$$|a\rangle \rightarrow 2^{-(n-1)/2} \sum_{y_1+y_2+\dots+y_n \equiv a} |y_1, \dots, y_n\rangle. \quad (7.4)$$

(Galindo, nd)

## The Complete Shor Code

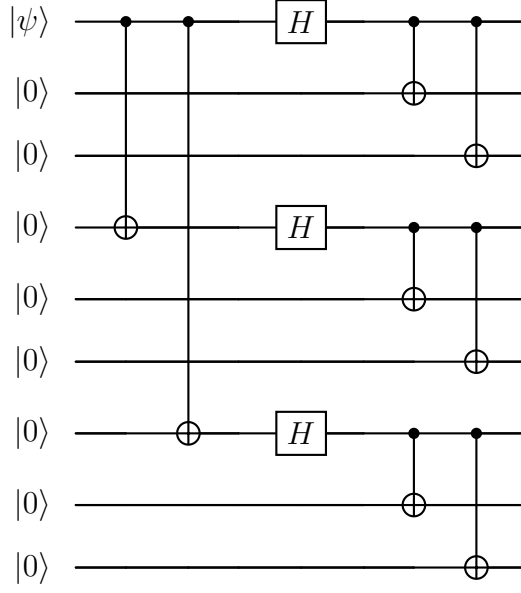
By concatenating both codes, we obtain the complete Shor code (Kitaev et al., 2002):

$$V = (V_r^z)^{\otimes r} \circ V_r^x : \mathcal{B} \rightarrow \mathcal{B}^{\otimes r^2}. \quad (7.5)$$

To visualize this concretely, it is helpful to expand the last expression into a matrix of  $r^2$  qubits grouped by rows:

$$|a\rangle \rightarrow 2^{-(r-1)/2} \sum_{y_1+y_2+\dots+y_r \equiv a} \begin{pmatrix} y_1 & y_1 & \dots & y_1 \\ y_2 & y_2 & \dots & y_2 \\ \vdots & \vdots & \ddots & \vdots \\ y_r & y_r & \dots & y_r \end{pmatrix}. \quad (7.6)$$

This can be represented by the following circuit in Figure 7.1, where a concrete example of the Shor code with 9 qubits is portrayed.



**Figure 7.1:** Encoding circuit for the Shor code with  $n = 3$ . This is a simple example demonstrating the different structural blocks explained previously (Galindo, nd).

## Error Detection and Code Distance

For a Shor code where  $n = r$ , any error  $f$  with a weight  $|f| < r$  can be detected. This can be shown using a slight simplification of the Knill-Laflamme conditions (Kitaev et al., 2002). For an error  $f$ , we evaluate:

$$\begin{aligned}\langle \varepsilon_1 | f | \varepsilon_0 \rangle &= 0 \\ \langle \varepsilon_1 | f | \varepsilon_1 \rangle &= \langle \varepsilon_0 | f | \varepsilon_0 \rangle.\end{aligned}$$

instead of the full Hermitian matrix. Both formulations are equivalent.

$$f^{(x)} \neq 0$$

Given that  $0 < |f^{(x)}| < r$ , at least one component must differ when the error is applied. Because this weight is insufficient to flip an entire row, applying this operation always yields 0:

$$\forall a, b \in \{0, 1\}, \quad \langle \varepsilon_a | f | \varepsilon_b \rangle = 0. \quad (7.7)$$

which immediately satisfies both conditions.

### $f^{(x)} = 0$ (Only phase errors)

In this case, only the phase changes. Due to phase kickback, the effect of the error  $f$  on the code states can be simplified to:

$$f |\psi\rangle = (-1)^{\sum_j \lambda_j y_j} |\psi\rangle. \quad (7.8)$$

where the vector  $\lambda$  represents the phase-flip operations for each column. For the general case:

$$\langle \varepsilon_a | f | \varepsilon_a \rangle = 2^{-(r-1)} \sum_{y_1 + y_2 + \dots + y_r \equiv a} (-1)^{\sum_{j=1}^r \lambda_j y_j}. \quad (7.9)$$

There are three possible cases:

- 1**  $\lambda = \vec{0}$ : The operation acts exactly as the identity over the code subspace.
- 2**  $\lambda = \vec{1}$ : This requires a phase flip in all  $r$  columns, meaning the weight of the error would be  $r$ . This contradicts our initial assumption that  $|f| < r$ .
- 3**  $\lambda$  is a non-trivial combination. By the orthogonality of characters, we have:

$$\sum_{y_1 + y_2 + \dots + y_r \equiv a} (-1)^{\lambda \cdot y} = 0. \quad (7.10)$$

Thus, any error affecting fewer than  $r$  qubits can be detected by the Shor code (Galindo, nd).

## Concrete Example: The Original 9-Qubit Shor Code

As shown in Figure 7.1, the Shor code with  $n = 3$  is the first code described to correct any error in a single qubit. In this case, the state  $|0\rangle$  can be summed by

the vectors with even parity  $v \in \{(0, 0, 0), (1, 1, 0), (1, 0, 1), (0, 1, 1)\}$  (Galindo, nd), making the encoded state:

$$|\varepsilon_0\rangle = \frac{1}{2} \left( \begin{vmatrix} 000 \\ 000 \\ 000 \end{vmatrix} + \begin{vmatrix} 111 \\ 111 \\ 000 \end{vmatrix} + \begin{vmatrix} 111 \\ 000 \\ 111 \end{vmatrix} + \begin{vmatrix} 000 \\ 111 \\ 111 \end{vmatrix} \right). \quad (7.11)$$

Additionally, the state  $|1\rangle$  is encoded by the vectors with odd parity, meaning  $v \in \{(1, 0, 0), (0, 1, 0), (0, 0, 1), (1, 1, 1)\}$  (Galindo, nd), making it:

$$|\varepsilon_1\rangle = \frac{1}{2} \left( \begin{vmatrix} 111 \\ 000 \\ 000 \end{vmatrix} + \begin{vmatrix} 000 \\ 111 \\ 000 \end{vmatrix} + \begin{vmatrix} 000 \\ 000 \\ 111 \end{vmatrix} + \begin{vmatrix} 111 \\ 111 \\ 111 \end{vmatrix} \right). \quad (7.12)$$

The distance of this code is, as expected,  $d = 3$ , allowing it to correct errors in 1 qubit. These errors can be divided into  $X$  and  $Z$  errors, or a combination of both, which will trigger these options (Galindo, nd):

- **Bit-flip error:** This would change a single qubit in a row, and by simple majority, as explained in the repetition code, it could be detected and corrected.
- **Phase-flip error:** By phase kickback, the phase of the whole row would change the overall phase. This can be tested by checking the parity with the other blocks to detect the change in phase and correct it.

This code is not the smallest code that can correct a general error in a single qubit. As discussed before, the Hamming bound for such a condition is 5 qubits. However, it is a very important and interesting code because it also fulfills the requirements for a stabilizer code, making it much more efficient than other competitors.

## Shor Code as a Stabilizer Code

An important property of the Shor code is that it is a stabilizer code (Kitaev et al., 2002). For the general Shor code, we can explicitly construct the stabi-

lizers (Galindo, nd). Beginning with Equation 7.6, we can define two groups of stabilizers.

## Internal Stabilizers

For every row  $j$  and two consecutive qubits within that row, we define the operator:

$$M_{jk}^{(z)} = \sigma_{jk}^{(z)} \sigma_{j(k+1)}^{(z)}. \quad (7.13)$$

(Galindo, nd)

For every pair  $j, k$ , the equation  $M_{jk}^{(z)} |\varepsilon\rangle = |\varepsilon\rangle$  holds because, as mentioned earlier, the rows consist of identical qubits; thus, the phase introduced by the  $\sigma_z$  operations cancels out.

## External Stabilizers

For every row  $j$  and its adjacent row  $j + 1$ , where  $j \in \{1, 2, \dots, r - 1\}$ , we can define the operation:

$$M_j^{(x)} = \prod_{k=1}^r \sigma_{jk}^x \sigma_{(j+1)k}^x. \quad (7.14)$$

(Galindo, nd)

This operation is a stabilizer because applying it is equivalent to the mapping:

$$|y_k, \dots, y_k\rangle \rightarrow |(y_k \oplus 1), \dots, (y_k \oplus 1)\rangle.$$

Applying it to rows  $j$  and  $j + 1$ , and recalling that every logical state is a superposition of all configurations  $(y_1, \dots, y_r)$  with a global parity  $\sum_i y_i \equiv a$ , we get:

$$\begin{aligned} (y_1 + y_2 + \dots + y_{j-1}) + (y_j \oplus 1) + (y_{j+1} \oplus 1) + (y_{j+2} + \dots + y_r) &= \sum_i^r y_i + 2 \pmod{2} \\ &= \sum_i^r y_i = a. \end{aligned}$$

This demonstrates that the operation  $M_j^{(x)}$  acts as a stabilizer for the Shor code encoding.

## Dimension of the Code and Verification of the Symplectic Condition

The combination of these stabilizers generates the group that defines the Shor code. Its dimension is:

$$\dim(SC) = 2^{r^2 - (r^2 - 1)} = 2. \quad (7.15)$$

This is consistent with encoding a single logical qubit, as previously explained.

We can verify the commutation relations between the defined operators by analyzing the following cases:

- **Two internal stabilizers  $M_{j_1 k_1}^{(z)}$  and  $M_{j_2 k_2}^{(z)}$ :** Since this type of operation uses exclusively  $Z$  gates (represented by  $(0, 1)$  in symplectic notation) and deviates from the identity operation only on specific qubits, it holds that  $\alpha = 0$  for every operator in  $M_{jk}$ . Therefore:

$$\omega(M_{j_1 k_1}^{(z)}, M_{j_2 k_2}^{(z)}) = \sum_i (0 \cdot \beta'_i - 0 \cdot \beta_i) = 0. \quad (7.16)$$

- **Two external stabilizers  $M_{j_1}^{(x)}$  and  $M_{j_2}^{(x)}$ :** These operations have the  $X$  gate as their only non-identity component. In symplectic notation,  $X = (1, 0)$ , making  $\beta = 0$  for every operation in the system, which results in:

$$\omega(M_{j_1}^{(x)}, M_{j_2}^{(x)}) = \sum_i (\alpha_i \cdot 0 - \alpha'_i \cdot 0) = 0. \quad (7.17)$$

- **One internal stabilizer  $M_{jk}^{(z)}$  and one external stabilizer  $M_\ell^{(x)}$ :** There are two possibilities. The first is  $j \neq \ell \wedge j \neq (\ell + 1)$ . In this case, the operations are disjoint, and therefore they commute trivially ( $\omega = 0$ ).

The second possibility is a collision in a single row:  $j = \ell \vee j = (\ell + 1)$ . In this case,  $M_{jk}$  applies  $Z$  to two qubits in a row where  $M_\ell$  applies  $X$ . The overlap between the operations acts on exactly 2 qubits, which evaluates to 0 over the modulo-2 field.



It is important to note that the Shor code specifically has the trivial solution  $\mu = 0$  for the sign operation. This is because the stabilizers operate on different columns or rows; therefore, when applied in conjunction, they yield independent results ( $\tilde{\omega} = 0$ ), making it unnecessary to add a sign function (Galindo, nd).



# Conclusions

This thesis established the fundamental structure of quantum error-correcting codes. By focusing on mathematical structure and algebraic consistency, it addressed the limitations imposed by quantum theory, such as the no-cloning theorem and the continuous nature of errors. In doing so, it demonstrated the efficacy of encoding information into larger Hilbert subspaces to protect it from environmental decoherence.

First, defining and treating errors as quantum operations over the system and its environment laid the necessary groundwork. In particular, it was explained why the focus is placed on independent qubit errors and how any continuous error can be decomposed into a basis of Pauli operations, ensuring that correcting these discrete errors automatically corrects the continuous ones. Building upon this foundation, the core structure of a quantum code was defined, introducing the Knill-Laflamme conditions as the definitive mathematical criteria for successful error recovery. Additionally, key theoretical concepts, such as the quantum Hamming bound and degenerate codes, were explored.

The core of this work centered on the stabilizer formalism and its corresponding codes, which serve as the primary framework for efficiently constructing and analyzing quantum error-correcting codes. To this end, it was initially necessary to examine classical linear codes, which act as the classical analogue to stabilizer codes, particularly regarding the use of parity-check matrices for efficient error detection. Following this, the study of the symplectic group and the derivation of extended symplectic operations—which can be systematically constructed in quan-

tum circuits—demonstrated how this algebraic structure functions analogously to classical parity checks, enabling efficient syndrome measurement.

Finally, the theoretical framework was applied to two concrete examples. First, the quantum repetition code was analyzed using the Knill-Laflamme conditions, evaluating its fidelity and illustrating its inherent inability to correct phase-flip errors. Second, the general Shor code was examined, successfully deriving its necessary stabilizers to prove that it functions as a valid stabilizer code capable of correcting arbitrary single-qubit errors.

In conclusion, the rigorous development of these theoretical foundations provides a solid understanding of quantum error-correcting codes. Moreover, it equips the reader with the necessary analytical tools to design and construct new fault-tolerant codes utilizing the stabilizer formalism.

# Bibliography

- AbuGhanem, M. (2025). IBM quantum computers: evolution, performance, and future directions. *The Journal of Supercomputing*, 81:687.
- Calderbank, A. R. and Shor, P. W. (1996). Good quantum error-correcting codes exist. *Physical Review A*, 54(2):1098–1105.
- Galindo, C. (n.d.). Códigos cuánticos. Lecture notes. Unpublished manuscript, obtained through private communication.
- Gottesman, D. (1998). Theory of fault-tolerant quantum computation. *Physical Review A*, 57(1):127–137.
- Kaye, P., Laflamme, R., and Mosca, M. (2006). *An Introduction to Quantum Computing*. Oxford University Press, Oxford, UK.
- Kitaev, A. Y., Shen, A., and Vyalij, M. N. (2002). *Classical and Quantum Computation*, volume 47 of *Graduate Studies in Mathematics*. American Mathematical Society.
- Nielsen, M. A. and Chuang, I. L. (2010). *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary edition edition.

